

Домашнее задание 4.

Dockerfile по правилам

Автор: Вольхин Сергей Александрович, группа Р2

В задании я собрал небольшой FastAPI-сервис с моделью sklearn, упаковал его в двухстадийный Docker-образ, запустил без root-прав в docker-compose с изоляцией БД в приватной сети, написал манифесты Kubernetes и задеплоил контейнер в Yandex Cloud как бессерверный.

1. Dockerfile

Ниже полный листинг. Я сделал две стадии: в builder ставлю зависимости в отдельный venv с кешем pip (через `--mount=type=cache`), а в runtime копирую только готовое окружение и папку app. Запускается всё из-под обычного пользователя appuser с UID 1001, добавил HEALTHCHECK и EXPOSE на порт 8000.

```
# syntax=docker/dockerfile:1.6
# ----- Stage 1: builder -----
FROM python:3.11-slim AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    PIP_NO_CACHE_DIR=0

WORKDIR /build

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:${PATH}"

COPY requirements.txt ./

RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --upgrade pip==24.2 && \
    pip install -r requirements.txt

# ----- Stage 2: runtime -----
FROM python:3.11-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PATH="/opt/venv/bin:${PATH}"

RUN groupadd --system --gid 1001 appgroup && \
    useradd --system --uid 1001 --gid appgroup --no-create-home --shell /usr/sbin/nologin appuser

WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
COPY --chown=appuser:appgroup app/ ./app/

USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/health', timeout=3).status == 200 else 1)"

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

2. Многостадийная сборка

Смысл разделения простой: в финальный образ не должны попадать компиляторы, кеш pip и прочий мусор, который остаётся после установки зависимостей. У меня в runtime лежат только сам Python, /opt/venv с пакетами и папка app. По размеру это выглядит так: однослойная сборка раньше весила примерно 1.2 ГБ, двухстадийная получается около 180 МБ на python:3.11-slim. Разница в 6-7 раз, и она хорошо видна даже в docker images.

Флаг `--mount=type=cache,target=/root/.cache/pip` ускоряет повторные сборки. BuildKit между билдами переиспользует уже скачанные wheel-файлы, для CI это довольно важно. Чтобы это вообще заработало, в первой строке Dockerfile стоит

`syntax=docker/dockerfile:1.6`, без этой директивы монтирование кеша просто не работает.

3. docker-compose.yml

В compose у меня два сервиса: app (сам FastAPI) и db (postgres:16-alpine). Сетей тоже две. backend с флагом internal: true - в ней сидят оба сервиса, наружу она не выходит. frontend подключена только к app, на ней проброшен порт 8000 на хост. У сервиса db нет секции ports, поэтому снаружи хоста до базы не достучаться, весь трафик идёт через app.

Том postgres_data именованный, после docker compose down данные не пропадают. У app включил restart: unless-stopped и depends_on с condition: service_healthy, то есть FastAPI не стартует, пока pg_isready не скажет, что postgres готов принимать соединения. Без этой проверки на старте бывают гонки: приложение поднимается быстрее базы и падает с ошибкой подключения.

```
name: hw4_docker

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    image: hw4-app:latest
    container_name: hw4_app
    restart: unless-stopped
    env_file:
      - .env
    environment:
      DATABASE_URL: ${DATABASE_URL}
      APP_PORT: "8000"
    ports:
      - "8000:8000"
    networks:
      - backend
      - frontend
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c",
        "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/health', timeout=2)
      interval: 30s
      timeout: 3s
      retries: 3
      start_period: 10s
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 128M

  db:
    image: postgres:16-alpine
    container_name: hw4_db
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
```

```
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: 512M
      reservations:
        cpus: "0.25"
        memory: 128M

networks:
  backend:
    driver: bridge
    internal: true
  frontend:
    driver: bridge

volumes:
  postgres_data:
    driver: local
```

Файл `.env.example` (переменные окружения, сам `.env` в репозиторий не коммитится):

```
DB_USER=appuser
DB_PASSWORD=
DB_NAME=appdb
DATABASE_URL=postgresql://appuser:@db:5432/appdb
```

4. Границы памяти и CPU

В `deploy.resources` я задал верхнюю планку (`limits`: cpu 1.0, память 512M) и нижнюю, зарезервированную (`reservations`: cpu 0.25, память 128M). Те же самые числа лежат в манифесте Kubernetes как `requests` и `limits` у контейнера. Сделал это специально, чтобы поведение в `compose` и в `k8s` совпадало один в один.

Зачем вообще нужны лимиты. Во-первых, один упавший в OOM сервис не должен утащить за собой остальные на той же машине. Во-вторых, без `requests` планировщик Kubernetes не знает, на какую ноду положить под, ему просто нечего сравнивать. И в-третьих, в `managed k8s` счёт выставляют по зарезервированным ресурсам, так что лимиты напрямую влияют на стоимость.

5. Kubernetes-деплой

В `deploy.yaml` три ресурса, разделённые ---.

ConfigMap `hw4-app-config` содержит несекретные переменные: имя базы, порт приложения, уровень логирования. Контейнер подтягивает их через `envFrom.configMapRef`. Пароль от БД в настоящем проекте я бы, конечно, вынес в `Secret`, но для учебного задания ConfigMap достаточно.

Deployment `apps/v1` с двумя репликами, стратегия `RollingUpdate` и `maxUnavailable: 0`, то есть при обновлении ни одна реплика не уходит в даунтайм. Ресурсы совпадают с лимитами из `compose`. Probes две разные: `livenessProbe` перезапускает под, если приложение зависло, а `readinessProbe` убирает под из балансировки, пока он ещё не прогрелся. Нужно это, чтобы трафик не шёл на готовящийся инстанс.

Service типа `ClusterIP` даёт стабильный внутренний адрес в кластере: `port 80` проксируется в `targetPort 8000`, селектор по метке `app: hw4-app`. Наружу кластера трафик обычно пускают через Ingress или Service типа `LoadBalancer`, но для учебного деплоя я проверял всё через `kubectl port-forward`.

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: hw4-app-config
  labels:
    app: hw4-app
data:
  APP_PORT: "8000"
  DB_NAME: "appdb"
  LOG_LEVEL: "info"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hw4-app
  labels:
    app: hw4-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hw4-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hw4-app
    spec:
      containers:
        - name: app
          image: localhost:5000/hw4-app:latest
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          envFrom:
            - configMapRef:
                name: hw4-app-config
          resources:
            requests:
```

```

        cpu: "250m"
        memory: "128Mi"
    limits:
        cpu: "1000m"
        memory: "512Mi"
    livenessProbe:
        httpGet:
            path: /health
            port: http
        initialDelaySeconds: 15
        periodSeconds: 20
        timeoutSeconds: 3
        failureThreshold: 3
    readinessProbe:
        httpGet:
            path: /health
            port: http
        initialDelaySeconds: 5
        periodSeconds: 10
        timeoutSeconds: 2
        failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata:
  name: hw4-app
  labels:
    app: hw4-app
spec:
  type: ClusterIP
  selector:
    app: hw4-app
  ports:
    - name: http
      port: 80
      targetPort: 8000
      protocol: TCP

```

6. Итоговые выводы

Самой приятной частью оказалась многостадийная сборка. Буквально четыре строки COPY --from=builder, и финальный образ сжимается раз в 6-7 без потери функциональности. Хорошо видно даже в docker images.

Больше всего я провозился с healthcheck для postgres. Сначала собрал его на psql -с 'SELECT 1', но контейнер успевал пройти проверку до того, как postgres действительно прогрелся. FastAPI стартовал и падал, потому что база ещё не принимала соединений. После перехода на pg_isready и добавления start_period: 10s проблема ушла.

Если сравнивать с ручной выкаткой на VPS (scp, systemd, nginx руками), то Docker с Kubernetes даёт повторяемость и декларативность почти даром. Все YAML лежат в git и являются единственным источником правды, никакого дрефта конфигурации. С другой стороны, k8s действительно сложный: Deployment, ReplicaSet, Pod, Service, Ingress, ConfigMap, Secret, PVC - для небольшого проекта этих сущностей явно многовато, и для двух-трёх сервисов честно хватает docker-compose.

Логичное продолжение - описать сам кластер и его окружение в Terraform или OpenTofu (как в ДЗ 3). Тогда и инфраструктура, и приложения становятся IaC-артефактами, а вся цепочка от кода до пода воспроизводится с нуля одной командой.